

Checklist: Ways to increase the security of JWT-based implementations

TO BEGIN WITH

1. Understand what you're using: decide whether you need JWS (Signature) or JWE(Ebcrypted), choose suitable algorithms, and analyze how they work (at least at a basic level — e.g., HMAC, public/private key). Carefully verify the library you chose for using JWT. Is there already a better, simpler mechanism that might work for your case?
-

KEYS

2. Use **sufficiently complex** symmetric/asymmetric keys.
 3. Have a **plan** for what to do in case one of the keys is **compromised** (leaked).
 4. Store keys in a secure way (not in code or committed to repositories!).
-

SIGNATURE

5. Do not allow the sender to define the signing algorithm (preferably enforce a fixed one on the server side).
 6. Make sure your implementation does **not accept the none algorithm**.
 7. Make sure your implementation does **not accept an empty signature** (i.e., verify it is checked).
 8. If you use JWE, verify the safety of both the algorithms and their implementation.
 9. Differentiate between `verify()` and `decode()`. In other words — make sure you're **really verifying the signature**.
-

GENERAL RULES

10. Make sure the token generated in one place **cannot be reused elsewhere**.
 11. Make sure **debug mode is turned off** and cannot be re-enabled with a simple trickAvoid sending tokens in the **URL** (it's not secure — they can end up in web server logs).
-

PAYLOAD

13. Make sure you don't store **sensitive data** in the JWS payload.
14. Protect yourself against **replay attacks** (by using `jti`, etc.).
15. Tokens should have **short expiration times** (e.g., via `exp`, `iat`, etc.). Check if your implementation properly validates expiration.
16. Think about whether you need functionality that **revokes tokens** (JWT doesn't do this by design, but there are ways to implement such a feature).

LIBRARIES

17. Carefully read the documentation of the library you're using.
 18. Check for known vulnerabilities in the library you're using (e.g., on cvedetails.com or the library's website).
 19. Make sure your previous projects don't use a vulnerable version of the library. Check whether new bugs have been reported in the library (they might appear months after deployment).
 20. Track new vulnerabilities in JWT libraries. A vulnerability discovered in another project might also exist in your project — if the same library is used in a similar way.
-